

VULNERABILITY ASSESSMENT REPORT

Contents

1. EXECUTIVE SUMMARY	3
2. METHODOLOGY	5
2.1 Assessment Approach	5
2.2 Tools Used	5
2.3 Scope	5
2.4 Process	5
3. SUMMARY OF FINDINGS	6
3.1 Overall Statistics	6
3.2 Vulnerabilities by Component	6
3.3 Top Critical Findings Summary	6
4. DETAILED FINDINGS & REMEDIATION	7
5. ADDITIONAL HIGHSEVERITY FINDINGS	15
5.1 OpenSSL Vulnerabilities (libssl3t64 3.5.6)	15
5.2 Multer Denial of Service Vulnerabilities (multer 1.4.5lts.2	15
5.3 Undici HTTP Client Vulnerabilities (undici 5.29.0)	15
5.4 SanitizeHTML XSS Vulnerabilities (sanitizehtml 1.4.2)	15
6. RECOMMENDATIONS	16
6.1 Immediate Actions (Priority 1 - Critical)	16
6.2 ShortTerm Actions (Priority 2 High, within 1 week)	16
6.3 LongTerm Actions (Priority 3 Medium, within 1 month)	16
7. CONCLUSION	17
APPENDIX A: TOOLS AND VERSIONS	17

Image: bkimminichjuiceshop:latest (v20.2.0)

Date of Assessment: September 18, 2026

Assessor: Yuvan Thiruchelvam

Tool Used: Trivy v0.74.0

1. EXECUTIVE SUMMARY

On September 18, 2026, a comprehensive container vulnerability assessment was conducted against a local Docker instance of the OWASP Juice Shop application using Trivy v0.74.0. The assessment examined the container image's operating system packages, application dependencies, and embedded secrets to identify security weaknesses that could be exploited by malicious actors.

The assessment revealed a critical security posture with a total of 134 vulnerabilities identified across the container image, including 8 CRITICAL and 45 HIGH severity findings. The most severe vulnerabilities include:

- ❖ Authentication bypass vulnerabilities in the `jsonwebtoken` library (CVSS 9.8)
- ❖ Cryptographic weaknesses in `cryptojs` (CVSS 9.1)
- ❖ Prototype pollution in `lodash` (CVSS 9.1)
- ❖ Command injection in `marsdb` (Critical)
- ❖ Arbitrary file read write in `decompress` (CVSS 9.1)
- ❖ Hardcoded RSA private keys embedded directly in application source code and InfrastructureasCode files

Additionally, hardcoded secrets were discovered, including RSA private keys in production code and Terraform configuration files.

If deployed in a production environment, these vulnerabilities would allow attackers to:

1. Bypass authentication and impersonate any user.
2. Execute arbitrary system commands on the server.
3. Read and write arbitrary files on the filesystem.
4. Steal sensitive cryptographic material.
5. Cause denial of service through multiple attack vectors.

Immediate remediation is strongly recommended, with priority given to updating vulnerable dependencies and removing all hardcoded secrets from the codebase.

2. METHODOLOGY

2.1 Assessment Approach

The assessment was conducted using a static container image analysis methodology, examining the Docker image layers and embedded dependencies without executing the application. This approach provides comprehensive visibility into the software supply chain and identifies vulnerabilities before deployment.

2.2 Tools Used

Trivy v0.74.0 (Aqua Security): Comprehensive container scanner for vulnerabilities, misconfigurations, and secrets

Docker Desktop: Container runtime environment

2.3 Scope

Target: `bkimminichjuiceshop:latest` (Docker image, 350 MB)

Components Analyzed:

- ❖ Operating System packages (Debian 13.6)
- ❖ Node.js application dependencies (npm packages)
- ❖ Embedded secrets and credentials
- ❖ InfrastructureasCode configurations

2.4 Process

1. Pulled the official OWASP Juice Shop Docker image.
2. Executed Trivy scan with default vulnerability database.
3. Analyzed results for OSlevel vulnerabilities (libc, OpenSSL, zlib).
4. Analyzed results for applicationlevel vulnerabilities (Node.js dependencies).
5. Identified hardcoded secrets in source code and configuration files.
6. Prioritized findings based on CVSS scores and exploitability.
7. Developed remediation recommendations.

3. SUMMARY OF FINDINGS

3.1 Overall Statistics

Category	Count
Total Vulnerabilities	134
CRITICAL	8
HIGH	45
MEDIUM	56
LOW	19
UNKNOWN	0
Exposed Secrets	6

3.2 Vulnerabilities by Component

Component	Type	Vulnerabilities
Debian 13.6 (OS)	os-pkgs	31
Node.js Dependencies	node-pkg	97
Source Code & IaC	secrets	6

3.3 Top Critical Findings Summary

ID	Vulnerability	Component	CVSS Score	Severity
1	JWT Verification Bypass	jsonwebtoken 0.1.0/0.4.0	9.8	CRITICAL
2	Command Injection	marsdb 0.6.11	N/A	CRITICAL
3	Prototype Pollution	lodash 2.4.2	9.1	CRITICAL
4	Weak Cryptographic PRNG	crypto-js 3.3.0	9.1	CRITICAL
5	Weak Password Hashing	crypto-js 3.3.0	9.1	CRITICAL
6	Arbitrary File Read/Write	decompress 4.2.1	9.1	CRITICAL
7	Authorization Bypass	express-jwt 0.1.3	9.1	HIGH
8	Hardcoded RSA Private Keys	Multiple files	N/A	HIGH

4. DETAILED FINDINGS & REMEDIATION

Finding 1: JWT Authentication Bypass via Algorithm Confusion

CVE ID: CVE20159235

CVSS v3.1 score: 9.8 (Critical)

Affected Component: `jsonwebtoken` versions 0.1.0 and 0.4.0

Location:

`juiceshopnode\_modulesjsonwebtokenpackage.json`

`juiceshopnode\_modulesexpressjwtnode\_modulesjsonwebtokenpackage.json`

Description:

The application utilizes severely outdated versions of the `jsonwebtoken` library (versions 0.1.0 and 0.4.0, released in 2014). These versions are vulnerable to an algorithm confusion attack where an attacker can bypass signature verification. By taking a token originally signed with an asymmetric public key (RS family) and resigning it with a symmetric algorithm (HS family) using the public key as the HMAC secret, an attacker can forge valid authentication tokens for any user, including administrators.

Business Impact:

Complete authentication bypass. An attacker could impersonate any user access restricted administrative functions and exfiltrate sensitive data without detection. This represents a critical failure of the application's access control mechanisms.

Remediation:

1. Immediately upgrade `jsonwebtoken` to version 9.0.0 or later, which enforces strict algorithm validation and removes support for the insecure `none` algorithm by default.

2. Audit all JWT verification logic to explicitly specify allowed algorithms:

```
jwt.verify(token, secretOrPublicKey, { algorithms: ['RS256'] });
```

3. Implement additional security controls such as token expiration and refresh mechanisms.

4. Conduct a security review of all authentication flows to ensure proper token validation.

Finding 2: Hardcoded RSA Private Keys in Source Code and Infrastructure

Rule ID: privatekey (AsymmetricPrivateKey)

Severity: HIGH

Affected Components:

`juiceshopbuildlibinsecurity.js` (Line 55)

`juiceshoplibinsecurity.ts` (Line 21)

`juiceshopinfrastructureterraformnetworking.tf` (Line 171)

`juiceshopterraformnetworking.tf` (Line 171)

Description:

Trivy's secret scanning engine detected multiple RSA private keys hardcoded directly within the application's TypeScript source code and Terraform InfrastructureasCode (IaC) configuration files. These keys are embedded in plaintext within the Docker image layers and can be extracted by anyone with access to the container.

Business Impact:

Exposure of private cryptographic keys compromises the entire Public Key Infrastructure (PKI). Attackers can use these keys to:

- ❖ Decrypt sensitive communications.
- ❖ Forge digital signatures.
- ❖ Impersonate the server.
- ❖ Bypass TLS/SSL protections.
- ❖ Access encrypted data stores.

This represents a catastrophic failure of secrets management and cryptographic key protection.

Remediation:

1. Immediate Actions:

- ❖ Rotate all compromised private keys immediately
- ❖ Remove all hardcoded secrets from the source code and IaC files
- ❖ Revoke any certificates or tokens signed with the exposed keys

2. Implement Secrets Management:

- ❖ Use a dedicated secrets management solution (e.g., HashiCorp Vault, AWS Secrets Manager, Azure Key Vault)
- ❖ Inject secrets at runtime via environment variables or secret mounting
- ❖ Never commit secrets to version control

3. Prevention:

- ❖ Integrate precommit hooks with tools like `gitsecrets`, `trufflehog`, or `gitleaks` to prevent future secret leakage
- ❖ Implement automated secret scanning in CICD pipelines
- ❖ Conduct regular code reviews focusing on secret management

Finding 3: Prototype Pollution Leading to Property Injection

CVE ID: CVE201910744

CVSS v3.1 score: 9.1 (Critical)

Affected Component: `lodash` version 2.4.2

Location: `juiceshopnode\_modulessanitizehtmlnode\_moduleslodashpackage.json`

Description:

The application depends on a severely outdated version of the `lodash` utility library (v2.4.2, released in 2014). The `defaultsDeep` function in this version is vulnerable to Prototype Pollution. An attacker can craft a malicious payload that modifies the `Object.prototype`, allowing them to inject or overwrite properties on all JavaScript objects within the application's runtime environment.

Business Impact:

- ❖ Prototype pollution can lead to:
- ❖ Denial of Service (application crashes)
- ❖ Property injection attacks
- ❖ Escalation to Remote Code Execution (RCE) if polluted properties are used in securitysensitive contexts (e.g., validation logic, access controls)
- ❖ Bypass of security controls and authentication mechanisms

Remediation:

1. Upgrade `lodash` to version 4.17.21 or later, which patches the prototype pollution vulnerability.
2. Conduct a dependency audit using `npm audit` to identify and update all transitive dependencies that rely on vulnerable versions of `lodash`.
3. Implement input validation and sanitization to prevent malicious object structures from reaching vulnerable functions.
4. Consider using `Object.freeze(Object.prototype)` in critical application paths to prevent prototype modification.

Finding 4: Cryptographic Weakness in Password Hashing and Random Number Generation

CVE IDs:

CVE202346233 (CVSS 9.1) Weak PBKDF2 implementation

CVE202671851 (CVSS 9.1) Weak pseudorandom number generator

Affected Component: `cryptojs` version 3.3.0

Location: `juiceshopnode\_modulescryptojspackage.json`

Description:

The `cryptojs` library (v3.3.0) contains two critical cryptographic vulnerabilities:

1. Weak PBKDF2 Implementation (CVE202346233): The library implements PBKDF2 (PasswordBased Key Derivation Function 2) with critically insecure defaults. It defaults to the SHA1 hash algorithm (considered cryptographically broken since 2005) and uses only a single iteration. Industry standards recommend a minimum of 600,000 iterations for SHA256 as of 2023. This implementation is approximately 1.3 million times weaker than the current industry standard.
2. Weak PRNG (CVE202671851): The library generates randomness using a custom variation of the MultiplyWithCarry pseudorandom number generator, seeded from `Math.random()`, instead of a cryptographically secure source. This produces effective search spaces small enough to enumerate commodity hardware.

Business Impact:

If these functions are used to:

- ❖ Hash user passwords: Attackers can perform offline bruteforce or dictionary attacks at extremely high speeds
- ❖ Generate authentication tokens or session IDs: Attackers can predict or enumerate valid tokens
- ❖ Generate cryptographic keys: Attackers can recover private keys and compromise encrypted data

Remediation:

1. Upgrade `cryptojs` to version 4.2.0 or later.
2. For password hashing: Migrate to modern, purposebuilt password hashing libraries such as:
 - `bcrypt` (recommended for most applications)
 - `argon2` (stateoftheart, winner of Password Hashing Competition)
 - `scrypt`
3. For random number generation: Use Node.js builtin `crypto.randomBytes()` or `crypto.randomUUID()` instead of applicationlevel PRNGs.
4. Audit all cryptographic operations to ensure they use current best practices and algorithms.

Finding 5: Command Injection via Unsanitized Input

Advisory ID: GHSA5mrrrgp6x4gr

Severity: CRITICAL

Affected Component: `marsdb` version 0.6.11

Location: `juiceshopnode\_modulesmarsdbpackage.json`

Description:

The `marsdb` database library is vulnerable to Command Injection. In the `DocumentMatcher` class, selectors using `\$where` clauses are passed directly to a JavaScript `Function` constructor without any sanitization or sandboxing. This allows an attacker to inject and execute arbitrary system commands on the host server when the function is evaluated.

Note: No fix is currently available for this vulnerability.

Business Impact:

Remote Code Execution (RCE). An attacker can achieve full control over the server, enabling them to:

- ❖ Exfiltrate all data.
- ❖ Install malware or backdoors.
- ❖ Pivot to internal networks.
- ❖ Cause irreversible damage to infrastructure.
- ❖ Use the compromised server as a launchpad for further attacks.

Remediation:

1. Immediate Actions:

- ❖ Since no patch is available, immediately remove or replace the `marsdb` dependency with a secure alternative (e.g., MongoDB official driver, SQLite, PostgreSQL).
- ❖ If the functionality is required, implement strict input validation and sanitization at the application layer to ensure no user-controlled input reaches the `\$where` clause.

2. Defense in Depth:

- ❖ Apply the principle of least privilege to the container runtime (run as non-root user).
- ❖ Implement network segmentation to limit the impact of a potential compromise.
- ❖ Deploy runtime security monitoring to detect suspicious command execution.

Finding 6: Arbitrary File Read Write via Crafted Archive Extraction

CVE ID: CVE202653486

CVSS v3.1 score: 9.1 (Critical)

Affected Component: `decompress` version 4.2.1

Location: `juiceshopnode\_modulesdecompresspackage.json`

Description:

The `decompress` package for Node.js extracts archives but fails to properly validate file paths and link targets. When extracting an archive to a directory, a crafted archive can read or write files outside that directory because:

- ❖ Hardlink and symlink entries are created without checking where targets point.
- ❖ Path containment used a string prefix comparison (easily bypassed).
- ❖ File modes failed to remove setuid, setgid, or sticky bits.

Business Impact:

Attackers can:

- ❖ Read sensitive files (e.g., `etcpasswd`, application secrets, database credentials)
- ❖ Write malicious files to arbitrary locations (e.g., cron jobs, SSH authorized_keys)
- ❖ Overwrite application binaries or configuration files
- ❖ Achieve Remote Code Execution by writing web shells or modifying application code

Remediation:

1. Upgrade to `@xhmikosrdecompress` versions 10.2.1 or 11.1.3, which contain fixes for this vulnerability.

2. If upgrading is not immediately possible:

- ❖ Avoid extracting untrusted archives.
- ❖ Implement additional path validation before extraction.
- ❖ Run extraction in isolated, temporary directories with restricted permissions.

3. Consider alternative archive extraction libraries with better security track records.

Finding 7: Authorization Bypass in expressjwt

CVE ID: CVE202015084

CVSS v3.1 score: 9.1 (High)

Affected Component: `expressjwt` version 0.1.3

Location: `juiceshopnode\_modulesexpressjwtpackage.json`

Description:

In `expressjwt` up to and including version 5.3.3, the `algorithms` entry to be specified in the configuration is not being enforced. When `algorithms` is not specified in the configuration, combined with libraries such as `jwksrsa` as the `secret`, it may lead to authorization bypass.

Business Impact:

Attackers can bypass authentication and access protected resources without valid credentials, leading to unauthorized data access and privilege escalation.

Remediation:

1. Upgrade `expressjwt` to version 6.0.0 or later.
2. Explicitly specify allowed algorithms in the configuration:

```
app.use(jwt({  
  secret: jwksRsa.expressJwtSecret({...}),  
  algorithms: ['RS256']  
}));
```

3. Review all JWT middleware configurations to ensure proper algorithm enforcement.

5. ADDITIONAL HIGHSEVERITY FINDINGS

The assessment identified numerous additional HIGH and MEDIUM severity vulnerabilities. Key findings include:

5.1 OpenSSL Vulnerabilities (libssl1.1.1 3.0.2)

CVE202014456 (CVSS 7.5): Denial of Service via unbounded memory growth in QUIC server

CVE202018798 (CVSS 7.5): Double free in QUIC server processing

CVE2020663072 (CVSS 7.5): Heap buffer overflow in CMS key unwrapping

Remediation: Upgrade OpenSSL to version 3.0.3 or later.

5.2 Multipart Denial of Service Vulnerabilities (multipart 1.4.5)ts.2)

- ❖ Multiple DoS vulnerabilities allowing attackers to crash the application through:
- ❖ Memory leaks from unclosed streams (CVE2020547935)
- ❖ Malformed multipart requests (CVE2020547944, CVE202057338)
- ❖ Deeply nested field names (CVE202065079)

Remediation: Upgrade to multipart 2.3.0 or later.

5.3 Undici HTTP Client Vulnerabilities (undici 5.29.0)

Multiple vulnerabilities include:

- ❖ WebSocket memory exhaustion (CVE2020612151, CVE202061526)
- ❖ HTTP header injection (CVE2020615157, CVE202061527)
- ❖ Request smuggling (CVE202061525)

Remediation: Upgrade to undici 6.28.0, 7.29.0, or 8.9.0 or later.

5.4 SanitizeHTML XSS Vulnerabilities (sanitizehtml 1.4.2)

Multiple CrossSite Scripting vulnerabilities allowing attackers to inject malicious scripts:

- ❖ CVE20161000237: XSS via nonrecursive sanitization.
- ❖ CVE201716016: XSS in certain scenarios with nonTextTags.
- ❖ CVE201925225: XSS via transformTags option.
- ❖ CVE202225887: ReDoS vulnerability.

Remediation: Upgrade to sanitizehtml 2.17.6 or later.

6. RECOMMENDATIONS

6.1 Immediate Actions (Priority 1 - Critical)

1. Update all CRITICAL vulnerabilities within 24 hours:

- ❖ jsonwebtoken → 9.0.0+
- ❖ lodash → 4.17.21+
- ❖ cryptojs → 4.2.0+
- ❖ decompress → 10.2.1+ or 11.1.3+
- ❖ Remove or replace marsdb (no fix available)

2. Remove all hardcoded secrets immediately:

- ❖ Rotate all exposed RSA private keys.
- ❖ Remove keys from source code and Terraform files.
- ❖ Implement proper secrets management.

3. Rebuild the Docker image with updated dependencies

6.2 ShortTerm Actions (Priority 2 High, within 1 week)

1. Update all HIGH severity vulnerabilities.
2. Implement automated dependency scanning in CICD pipeline.
3. Add precommit hooks for secret detection.
4. Conduct security review of authentication and authorization mechanisms.

6.3 LongTerm Actions (Priority 3 Medium, within 1 month)

1. Establish regular vulnerability scanning schedule (weekly).
2. Implement Software Composition Analysis (SCA) in development workflow.
3. Create dependency update policy and procedures.
4. Conduct penetration testing to validate remediation efforts.
5. Implement runtime application security monitoring.

7. CONCLUSION

The OWASP Juice Shop container image contains severe security vulnerabilities that violate secure coding practices and industry standards. The combination of outdated dependencies with known critical vulnerabilities, hardcoded cryptographic keys, and command injection flaws creates an unacceptable risk profile for production deployment.

The development team should prioritize fixing the CRITICAL and HIGH severity vulnerabilities before the application is considered for any production deployment. A follow up rescan should be conducted after patches are applied to verify the remediation.

Overall Risk Rating: CRITICAL

This assessment demonstrates the importance of:

- ❖ Regular dependency updates and vulnerability scanning.
- ❖ Proper secrets management practices.
- ❖ Secure software development lifecycle (SDLC) integration.
- ❖ Container security as part of DevSecOps practices.

APPENDIX A: TOOLS AND VERSIONS

Tool	Version	Purpose
Trivy	0.74.0	Container vulnerability scanning
Docker Desktop	Latest	Container runtime
OWASP Juice Shop	20.2.0	Target application